

TREE 1

PROBLEM 8

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 68
Question description A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries. Constraints $1 \leq n, q \leq 2 \cdot 10^5$ $1 \leq p_i \leq 10^9$ $1 \leq k \leq n$ $1 \leq x \leq 10^9$ $1 \leq a \leq b \leq 10^9$ Input The first input line contains two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \dots, n$. The next line has n integers $p_1, p_2, \dots, p_n$: each employee's salary. After this, there are q lines describing the queries. Each line has one of the following forms: ! k x: change the salary of employee k to x ? a b: count the number of employees whose salary is between $a \dots b$ Output Print the answer to each ? query.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 3 3 7 2 2 5 ? 2 3 ! 3 6 ? 2 3 EXPECTED OUTPUT 3 2	INPUT (STDIN) 5 3 31 17 12 12 15 ? 12 31 ! 13 6 ? 2 13 EXPECTED OUTPUT 5 3

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD int compare(const void *a, const void *b)	KEYWORD void update(int i, int n, int x)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 1	TOKEN COUNT 575	NLOC 65

SOURCE CODE:

```
#include <stdio.h>

int employee_wages[200005];

int main() {
    int total_staff, operation_count;
    if (scanf("%d %d", &total_staff, &operation_count) != 2) return 0;

    for (int i = 1; i <= total_staff; i++) {
        scanf("%d", &employee_wages[i]);
    }

    while (operation_count--) {
        char op_code;
        scanf("%c", &op_code);
        if (op_code == '!') {
            int emp_id, new_wage;
            scanf("%d %d", &emp_id, &new_wage);
            employee_wages[emp_id] = new_wage;
        } else if (op_code == '?') {
            int min_bound, max_bound;
            scanf("%d %d", &min_bound, &max_bound);
            int qualified = 0;
            for (int i = 1; i <= total_staff; i++) {
                if (employee_wages[i] >= min_bound && employee_wages[i] <= max_bound) {
                    qualified++;
                }
            }
            printf("%d\n", qualified);
        }
    }
    return 0;
}
```

OUTPUT:

```
5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3
3
2
```

```
5 3
31 17 12 12 15
? 12 31
! 13 6
? 2 13
5
3
```